

Mid Tearm

Pdf - 01

1. What is Microprocessors?

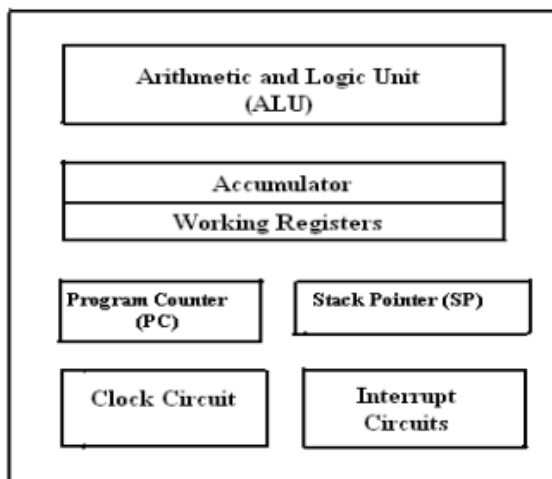
Ans: Microprocessor is the core of computer systems. A microprocessor is a multipurpose, programmable, clock-driven, register based electronic device that reads binary instructions from a storage device called memory.

2. Why do we need to learn Microprocessors and Microcontrollers?

Today there is no electronic gadget on the earth which is designed without a Microcontroller. Ex: communication devices, digital entertainment, portable devices etc...

- **Personal information products:** Cell phone, pager, watch, pocket recorder, calculator.
- **Laptop components:** mouse, keyboard, modem, fax card, sound card, battery charger
- **Home appliances:** Door lock, alarm clock, air conditioner, Tv remote etc.
- **Industrial equipment:** Temperature/pressure controllers, Counters, timers, RPM Controllers
- **Toys:** video games, cars, dolls, etc.

3. Explain Microprocessor with block diagram.



i. Arithmetic and Logic Unit (ALU) : It is the place where actual computations take place. Consist of circuits which performs arithmetic operations.

ii. Control Unit (CU): The coordination and control of a computer is the sole responsibility of the Control Unit.

iii. Registers: Registers are temporary storage locations inside microprocessor that holds data and address.

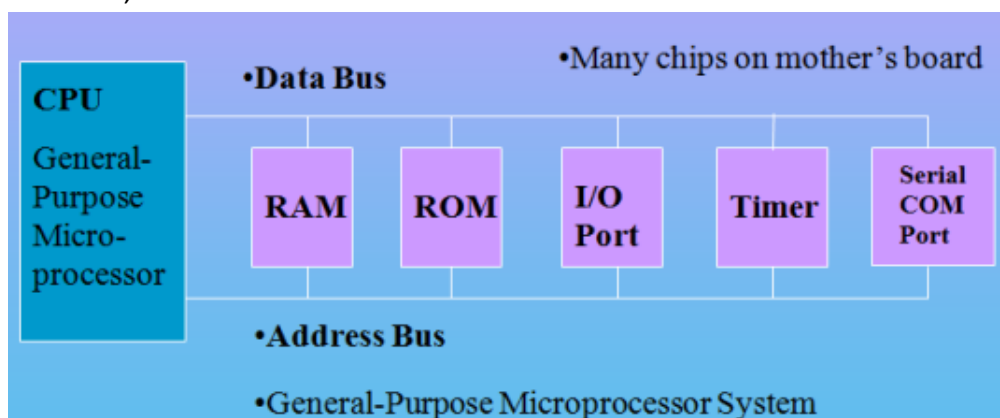
iv. Program counter (PC): It is commonly known as IP (Instruction Pointer) in 8086, is a register in microprocessor that contains the address of the next instruction to be executed.

v. Stack Pointer (SP): It is a small register that stores the address of the last program/instruction request in a stack.

Fig. Block diagram of a Microprocessor.

4. Microprocessor:

- CPU for Computers
- No RAM, ROM, I/O on CPU chip itself
- Example: Intel's x86, Motorola's 680x0



5. What is Microcontroller?

Ans: A microcontroller is a highly integrated single chip, which consists of on chip CPU (Central Processing Unit), RAM (Random Access Memory), EPROM/PROM/ROM (Erasable Programmable Read Only Memory), I/O (input/output) – serial and parallel ports, timers, interrupt controller.

For example: Intel 8051 is 8-bit microcontroller and Intel 8096 is 16-bit microcontroller.

6. Block Diagram of Microcontroller:

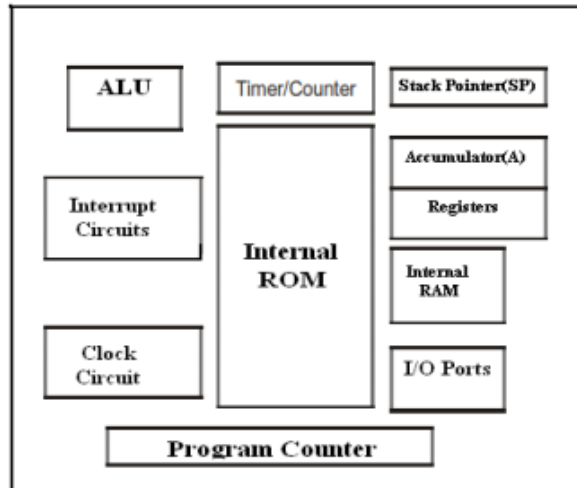


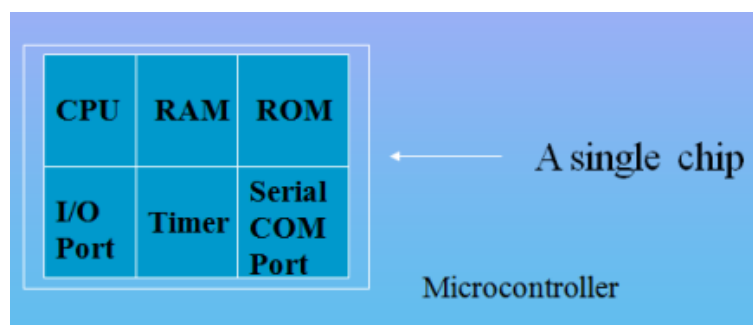
Fig. Block Diagram of a Microcontroller

7. Applications of Microcontrollers:

Cell phone, Pager, watch, calculator, video games, Alarm clock, air conditioner, Tv remote, Microwave oven, washing machines, robotic system, anti-lock braking system monitor.

8. Microcontroller:

- A smaller computer
- On-chip RAM, ROM, I/O ports...
- Example: Motorola's 6811, Intel's 8051, Zi log's Z8 and PIC



9. What is RISC (Reduced Instruction Set Computer)?

It stands for Reduced Instruction Set Computer. It is a type of microprocessor architecture that uses a small set of instructions of uniform length. These are simple instructions that are generally executed in one clock cycle. RISC chips are relatively simple to design and inexpensive. **Examples: SPARC, POWER PC etc.**

10. What is CISC (Complex Instruction Set Computer)?

It stands for Complex Instruction Set Computer. These processors offer the users, hundreds of instructions of variable sizes. CISC architecture includes a complete set of special-purpose circuits that carry out these instructions at a very high speed. CISC processors reduce the program size and hence lesser number of memory cycles are required to execute the programs. This increases the overall speed of execution.

Examples: Intel architecture, AMD

11. Distinguish with Microprocessor and Microcontroller.

Microprocessor	Microcontroller
CPU is stand-alone, RAM, ROM, I/O, timer is separate	CPU, RAM, ROM, I/O and timer are all on a single chip
General-purpose	Single-purpose (control-oriented)
Designer can decide on the amount of ROM, RAM and I/O ports.	Fixed amount of on-chip ROM, RAM, I/O ports
High processing power	Low processing power
Typically, 16/32/64 – bit	Typically, 8/16 bit
Typically, deep pipeline (5-20 stages)	Typically, single-cycle/two-stage pipeline

12. Distinguish with RISC and CISC.

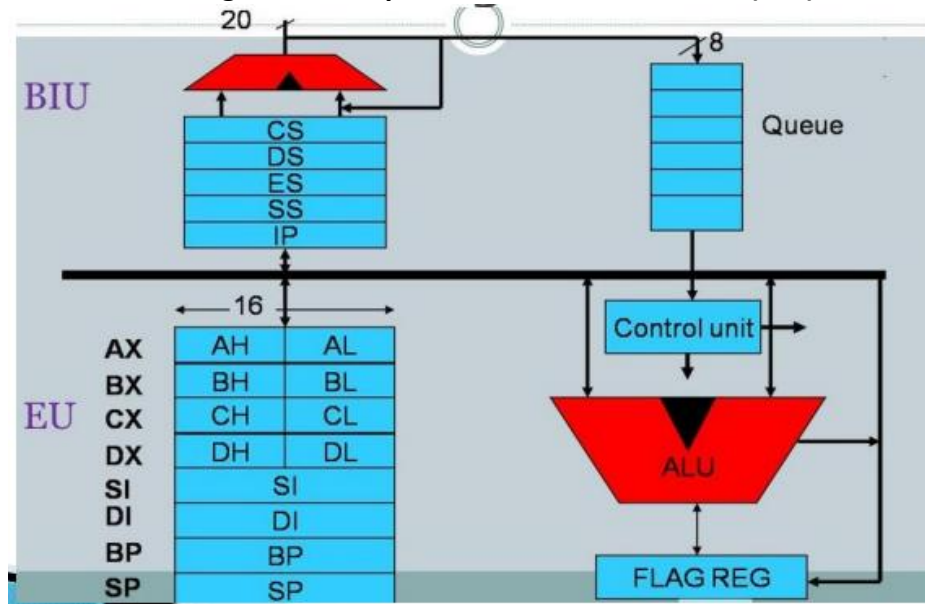
RISC	CISC
RISC stands for Reduced Instruction Set Computer. It's focus on software.	CISC stands for Complex Instruction Set Computer. It's focus on Hardware.
Uses only Hardwired control unit.	Uses both hardwired and microprogrammed control unit.
Can perform only Register to Register Arithmetic operations	Can perform REG to REG or REG to MEM or MEM to MEM
RISC is Reduced Instruction Cycle.	CISC is Complex Instruction Cycle.
Simple and limited addressing modes.	Complex and more addressing modes.
Code size is large	Code size is small
Here, addressing modes are less.	Here, addressing modes are more.

Pdf – 02

13. Explain the 8086 Microprocessor

- ✓ It is a 16-bit microprocessor
- ✓ It has 16-bit data bus
- ✓ It has 20-bit address bus
- ✓ 8086 CPU is divided into two independent functional parts.
- ✓ Bus interface unit (BIU)
- ✓ Execution unit (EU)

14. Draw the 8086 internal block diagram and explain the Bus Interface Unit(BIU) and Execution Unit(EU)



***Bus Interface Unit:

1. Instruction Queue-

- ❖ 6-byte queue
- ❖ Prefetch instructions (FIFO)

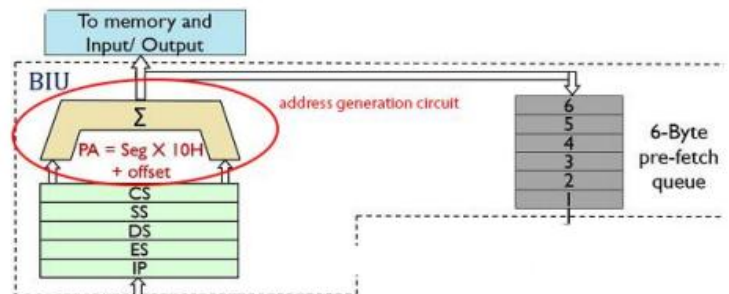
2. Segment Register-

Start address of a data. It has 4 segment registers that holds address of instruction and data in memory.

- Code Segment register
- Data Segment register
- Extra Segment register
- Stack Segment register

3. Instruction Pointer –

It's a 16 bit register that holds offset of the next instructions in the CS.



***Execution Unit in 8086:

This is an execution unit where instruction decoding and execution takes place.

1. General Purpose Register : Four 16 bit register divided into 2 parts low [8bit] and high [8bit] use for data handling.

- | | |
|--------|---------|
| i. AX | iii. CX |
| ii. BX | iv. DX |

2. Pointers register : contain offset address.

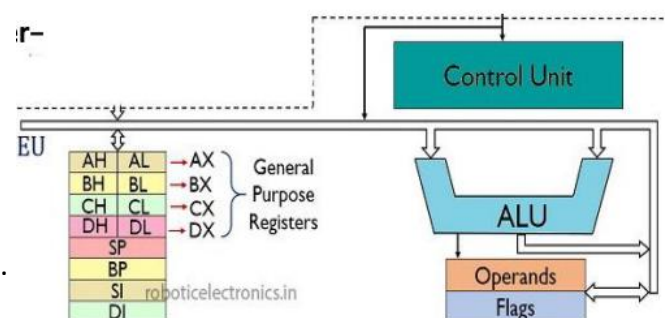
- SP(Stack pointer)- point to stack, top of stack segment.
 $PA = [SS] * 10H + [SP]$
- BP(Base pointer)- use to access random location of stack.
 $PA = [DS] * 10H + [BP]$

3. Index register:

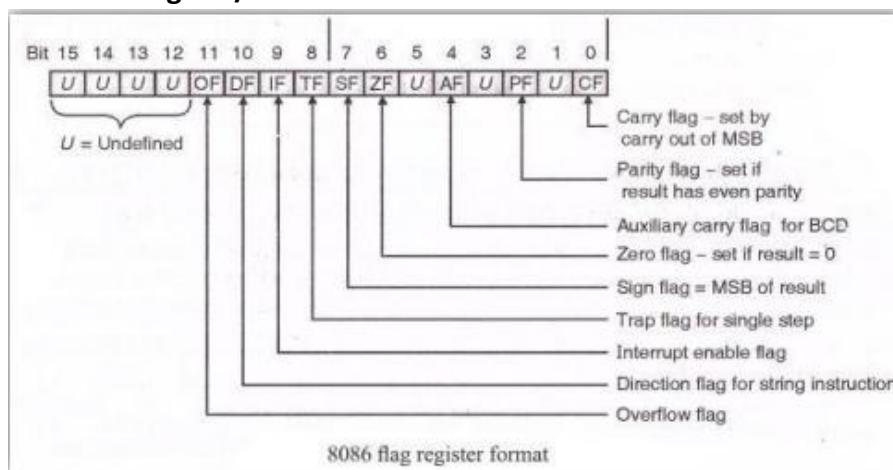
- DI (Destination Index)
- SI (Source Index)

4. Temporary register: Store temporary

5. ALU : Perform arithmetic and logical Operation.



15. Explain Flag registers with diagram/ format



The flag register is a 16-bit register in the Intel 8086 microprocessor that contains information about the state of the processor after executing an instruction. There are two types of flag register:

1) conditional/status and 2) control flag

1. Conditional/ Status flag:

- **Carry flag-** It is set whenever there is a carry or borrow out of the MSB (most significant bit) of a result. D7 bit for an 8-bit operation and D15 bit for a 16-bit operation.
- **Auxiliary carry flag-** this flag is set when ALU perform BCD operation of 8-bit and if result in a carry or borrow.
- **Parity flag-** It is set if the result has even parity. If parity is odd, PF is reset.
- **Zero flag-** This flag is set 1 when the result of ALU operation is zero else it is set to zero.
- **Sign flag-** It is set if the MSB of the result is 1.
- **Overflow flag-** It will be set if the result of a signed operation is too large to fit in the number of bits available to represent it.

2. Control Flag:

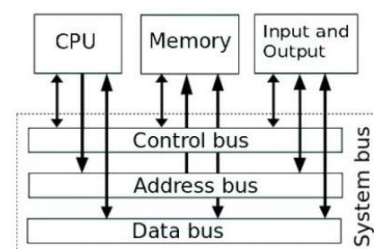
- **Trap flag-** It is used to set the trace mode i.e. start single stepping mode.
- **Interrupt flag-** this flag is used to enable or disable interrupt. It is set to 1 for interrupt enabled and set to 0 for interrupt disabled
- **Direction flag-** It is used in string operation.

Pdf – 3

16. Function of 8085 with diagram.

Ans:

- **Bit:** A bit is a single binary digit.
- **Word:** A word refers to the basic data size or bit size that can be processed by the arithmetic and logic unit of the processor. A 16-bit binary number is called a word in a 16-bit processor.
- **Bus:** A bus is a group of wires/lines that carry similar information.
- **System Bus:** The system bus is a group of wires/lines used for communication between the microprocessor.
- **Address Bus:** It carries the address, which is a unique binary pattern used to identify a memory location or an I/O port. For example, an eight-bit address bus has eight lines and thus it can address $2^8 = 256$ different locations.
- **Data Bus:** The data bus is used to transfer data between memory and processor or between I/O device and processor. For example, an 8-bit processor will generally have an 8-bit data bus and a 16-bit processor will have 16-bit data bus.
- **Control Bus:** The control bus carry control signals, which consists of signals for selection of memory or I/O device from the given address.



17. Functions of Control Unit of 8085.

Ans:

- **Data Bus:** Data bus carries data in binary form between microprocessor and other external units such as memory. Data bus is **bidirectional** in nature. The data bus width of 8085 microprocessor is 8-bit. 2^8 combination of binary digits and are typically identified as D0 – D7.

- **Address Bus:** The address bus carries addresses and is one-way bus from microprocessor to the memory or other devices. 8085 microprocessors contain 16-bit address bus and are generally identified as A0 - A15. the eight data bits (D0 – D7) and hence, they are bidirectional.

- **Control Bus:** The control bus carries control signals partly unidirectional and partly bidirectional. The following control and status signals are used by 8085 processor:

I. ALE (Address Latch Enable):
$$\text{ALE} = \begin{cases} 1 \text{ (Eable)} & \text{AD}_0\text{-AD}_7 = \text{A}_0\text{-A}_7 \text{ (Will carry only address).} \\ 0 \text{ (disable)} & \text{AD}_0\text{-AD}_7 = \text{D}_0\text{-D}_7 \text{ (Will carry only data).} \end{cases}$$

II. RD (active low output): when the RD = 0, it will do read operation.

III. WR (active low output): when the WR = 0, it will do write operation.

IV. IO/M : It is a signal that distinguishes between a memory operation and an I/O operation.

V. S1 and S0: These are status signals used to specify the type of operation being performed; they are listed in Table 1.

Table 1 Status signals and associated operations.

S1	S0	States
0	0	Halt
0	1	Write
1	0	Read
1	1	Fetch

Pdf – 4

18. Function of control Unit of 8086.

- **Data bus:** Carries data in binary form between microprocessor and other external units and vise-versa. The data bus width of 8086 microprocessor is 16-bit. 2^{16} combination of binary digits and are typically identified as D0 – D15.

- **Address Bus:** It contain 20 bit address bus and identified as A0-A19.

- **(AD0 –AD15) [Address/data bus]:** address bus are multiplexed with (D0-D15) data bus.

- **(A16/S3 –A19/S6) [Address/status bus]:** there are 4 address/status bus.

- **Control bus:** Carries control signals. The following control and status signals used in 8086 microprocessor.

Pin in max mode-

- BHE/S7- bus high enable is an active low signal, so it active when BHE= 0.
- RD- this is used for read operation when RD = 0.

○ $\overline{\text{MN}}/\overline{\text{MX}}$ - $\begin{cases} 1, \text{ maximum mode} \\ 0, \text{ minimum mode} \end{cases}$

Pin in min mode-

- **WR-** 0, then write operation active.
- **ALE-** address appears on (AD0-AD15).

○ $\overline{\text{IO}}/\text{M}$ - $\begin{cases} 0, \text{ I/O operation} \\ 1, \text{ memory operation} \end{cases}$

Pdf- 5

19. What is Addressing Modes?

Ans: The various formats of specifying the operands are called addressing modes.

20. Memory Addressing Modes.

In this addressing mode, one operand (source or destination) of an instruction must be an address of a memory location, which will be calculated by PA.

PA= [Segment] x 10H + Offset

Example: Offset or EA= [ARRAY] + [BX] + [SI] = 2AH + 5000H + 2000H = 702AH

21. Explain Six types of memory addressing mode.

i. Direct or Displacement Addressing mode: In this mode, the EA is directly given in the instruction.

Example: MOV AX, [0500]; EA=0500H, AX will get data from 0500H address.

ii. Register Indirect Addressing mode: In this mode, the EA is in SI, DI or BX, which is calculated by PA.

PA= [DS] x 10H+EA

Example: MOV AL, [SI]; Calculated PA=21200H is the address of memory location.

iii. Based Addressing mode: In this mode, the EA is calculated by adding the contents of BX or BP and a displacement (variable).

So, PA= [DS]x10H+EA

Example: MOV BL,ARRAY[BX]; Calculated PA=32105H is the address of memory location

iv. Indexed Addressing mode: In this mode, the EA is calculated by adding the contents of SI or DI and a displacement (variable).

So, PA= [DS]x10H+EA

Example: MOV AL,START[SI]; Calculated PA=12403H is the address of memory location

v. Based Indexed Addressing mode: In this mode, the EA is calculated by adding the contents of BX or BP, SI or DI and a displacement (variable).

So, PA= [DS]x10H+EA

Example: MOV CL, DISPLACEMENT[BX][SI]; Calculated PA=81A07H is the address of memory location

vi. String Addressing mode: This addressing mode is related to string instructions.

Example: MOVS B ; MOVS Move the string, B Byte
MOVS W ; MOVS Move the string, W Word

22. Math.

****Suppose, [SI]=0400H, [DI]=0500H, [DF]=1, [DS]=3000H, [ES]=5000H, [30400H]=02H, [50500H]=05H. Justify the following instruction:
MOVS B.**

Solution: In this addressing mode, the following steps will be followed:

- At first, the two physical addresses (PA1 and PA2) will be calculated:
 $PA1 = [DS] \times 10H + [SI]$ and $PA2 = [ES] \times 10H + [DI]$
- Assign the PAs values from the given values.
- The contents of PA1 will be moved into PA2.
- Write the revised values of PA1 and PA2 after the instruction will be executed.
- Finally, the revised values of SI and DI will be written according to the given value of DF.

Solve the following Home Works (HW):

Now, $PA1 = [DS] \times 10H + [SI] = 30400H$
 $PA2 = [ES] \times 10H + [DI] = 50500H$

Before instruction execution,

$PA1 = [30400H] = 02H$ and $PA2 = [50500H] = 05H$

After instruction execution,

$PA1 = [30400H] = 02H$ and $PA2 = [50500H] = 02H$

Before instruction execution,

$[SI] = 0400H$ and $[DI] = 0500H$

After instruction execution,

$[SI] = 0399H$ and $[DI] = 0499H$; Because $DF=1$

1. Calculate the Physical Addresses with comments of the following instructions:

i). MOV [BX],AL; ii). MOV BL,[SI]; where $[DS]=3000H$, $[BX]=250AH$ and $[SI]=0300H$.

2. Mention the name of addressing modes, find out PAs of the following instructions:

i). MOV AL,START[BX]; ii). MOV BETA[DI],CL and MOV AL,ARRAY[BX][SI]
Where $[DS]=2000H$, $[BX]=2A25H$, $[SI]=0700H$, $[DI]=0200H$, $[START]=02H$, $[BETA]=03H$ and $[ARRAY]=04H$.

3. Justify the below instruction with the name of addressing mode:

i). MOVS B

Assume the following data prior to execution of the above instruction:

$[SI]=0300H$, $[DI]=0400H$, $[DF]=0$, $[BX]=5000H$, $[DS]=2000H$, $[ES]=3000H$, $[20300H]=05H$, $[30400H]=09H$.

23. What is Assembly Language?

- **Assembly language** is a low-level language that helps to communicate directly with computer hardware. An assembler is used to convert assembly code into machine language.
- Assembly languages contain **mnemonic codes** that specify what the processor should do. Ex – ADD for addition, SUB for subtraction.

24. Syntax of Assembly Language.

Assembly language code is generally **not case sensitive**. but we use upper case to differentiate code from the rest of the text.

Statements:

Programs consist of statements, one per line. Each statement is either an **instruction**, which the assembler translates into machine code, or an **assembler directive**.

name	operation	operand(s)	comment
------	-----------	------------	---------

❖ An example of an instruction is :

START:	MOV CX,5		; initialize counter
--------	----------	--	----------------------

Here, the **name field** consists of the **label START**: The **operation** is **MOV**, the **operands** are **CX and 5**, and the comment is ; initialize counter.

25. Name Field.

Ans: i. The name field is used for instruction labels, procedure names, and variable names.

ii. Names can be from 1 to 31 characters long, and may consist of letters, digits, and special characters like ?, @, _, \$, % .

iii. Some Example of legal names : diu_25, diu@cse, .Test, Done?

iv. Some Example of illegal names : TWO WORDS, 2abc, A45.28, You&ME

26. Operation Field.

Ans: MOV, ADD, SUB

Final Term

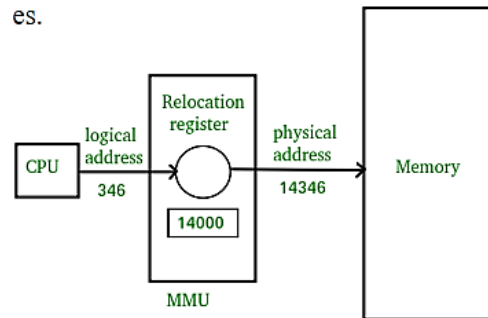
06. Logical Address and Physical Address

27. Define Logical Address.

Ans: A logical address, also known as a virtual address, is an address generated by the CPU during program execution.

28. Define Physical Address with figure.

Ans: A physical address is the actual address in the main memory where data is stored. Physical addresses are used by the Memory Management Unit (MMU) to translate logical addresses into physical addresses.



29. Compare Logical and Physical Address.

Logical Address	Physical Address
1. generated by CPU	1. location in a memory unit
2. User can view the logical address of a program.	2. User can never view physical address of program.
3. The user can use the logical address to access the physical address.	3. The user can indirectly access physical address but not directly.
4. Logical address can be change.	4. Physical address will not change.
5. virtual address.	5. real address.

30. Explain memory size for 8086.

Ans: •The **Intel 8086** microprocessor has a **20-bit address bus**.

•An address bus determines the range of memory addresses the processor can access.

With 20 address lines, the total number of unique memory locations that can be addressed is:

$$2^{20} = 1,048,576 \text{ bytes}$$

$$= 1 \text{ Megabyte (MB) bytes}$$

$$= 1 \text{ Megabyte (MB)}$$

07. Memory banking in 8086

31. Concept of memory banking in 8086 with figure and Table.

Ans:

- Each memory location can store only one byte of data.
- But the data bus of 8086 processor has 16 data lines.
- Thus the 8086 processor memory is divided into two banks where in each bank provides 8-bit data.
- Physically, memory is implemented as two independent 512Kbyte banks: the low(even) bank and the high (odd) bank.

- Data bytes associated with an even address (00000H-FFFFEH) reside in the low bank, and those with odd addresses (000001H-FFFFFH) reside in the high bank.
- Address bits A1 through A19 select the storage location that is to be accessed.
- Each of the memory banks provides half of the 8086's 16-bit data bus. The lower bank transfers bytes of data over data lines D0 through D7, while data transfers for a high bank use D8 through D15.

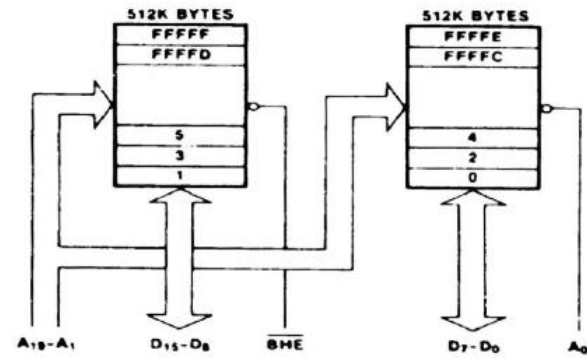


Table 1: Selection of odd and even memory bank.

$\overline{\text{BHE}}$	A0	Function
0	0	Choose both odd and even memory bank
0	1	Choose only odd memory bank
1	0	Choose only even memory bank
1	1	None is chosen

32. How can 8086 read 8-bit data from an even address?

Ans: Step:

- 1) Set $\overline{\text{BHE}} = 1$ and $A_0 = 0$.
- 2) This selects the **even address memory bank** (lower byte bank).
- 3) Calculate the physical address using:

$$\text{PA} = \text{DS} \times 10\text{H} + \text{Offset}$$
- 4) The 8086 reads the **8-bit data** from that even address.
- 5) The data is transferred to the **lower 8-bit register**, such as AL, BL, CL, or DL.

Example:

MOV AL, START[BX]

Given:

- DS = 2000H
- START = 04H
- BX = 1200H

We know that,

$$\text{PA} = \text{DS} \times 10\text{H} + (\text{START} + \text{BX})$$

$$\begin{aligned} \text{PA} &= 2000\text{H} \times 10\text{H} + (04\text{H} + 1200\text{H}) \\ &= 20000\text{H} + 1204\text{H} \\ &= 21204\text{H} \end{aligned}$$

Since, [21204H] = A9H is an even address, 8086 reads the 8-bit data from this address into AL.

33. How can 8086 read 8-bit data from an odd address?

Ans: Steps:

1. Set $\overline{\text{BHE}} = 0$ and $A_0 = 1$.
2. This selects the **odd address memory bank** (upper byte bank).
3. Calculate the physical address using:

$$\text{PA} = \text{DS} \times 10\text{H} + \text{Offset}$$
4. The 8086 reads the **8-bit data** from that odd address.
5. The data is transferred to the **higher 8-bit register**, such as AH, BH, CH, or DH.

Example:

MOV AH, ARRAY[SI]

Given:

- DS = 3000H
- ARRAY = 05H
- SI = 0300H

So, **PA = DS * 10H + (ARRAY + SI)**

$$\begin{aligned}\text{PA} &= 3000\text{H} \times 10\text{H} + (05\text{H} + 0300\text{H}) \\ &= 30000\text{H} + 0305\text{H} \\ &= 30305\text{H}\end{aligned}$$

Since [30305H] = 2BH is an odd address, it belongs to the **Odd/Upper Memory Bank**.

34. How can 8086 read 16-bit data from both even and odd addresses?

Ans: Step:

1. Set $\text{BHE} = 0$ and $\text{A}_0 = 1$.
2. Find out the two physical Address (PA1 and PA2).
3. **Finally, two 8-bit (8*2=16) data (2 bit hex data) will be moved from that memory to lower order by (AL, BL, CL, DL) and higher order byte of general purpose registers (AH, BH, CH, DH).**

Example:

MOV CX, DISPLACEMENT[SI]

Given:

- DS = 5000H
- SI = 0600H
- DISPLACEMENT = 0006H

So, **PA = DS * 10H + (SI + DISPLACEMENT)**

$$\begin{aligned}\text{PA} &= 5000\text{H} \times 10\text{H} + (0600\text{H} + 0006\text{H}) \\ &= 50000\text{H} + 0600\text{H} + 0006\text{H} \\ &= 50000\text{H} + 0600\text{H} + 0006\text{H} \\ &= 50606\text{H}\end{aligned}$$

PA 1 = 50606H -> even address

PA 2 = 50606+1 = 50607H -> odd address

Since, [50606H] = 8CH and [50607H] = 2DH . So that 8CH data will be moved from the even address 50606H to CL and 2DH data will be moved from the odd address 50607H to CH.

08. Microcontroller 8051

35. Define Microcontroller.

Ans: A microcontroller also called MCU (Microcontroller Unit) is a small computer on a single metal-oxide-semiconductor (MOS) integrated circuit (IC) chip. Microcontrollers are designed for embedded applications, in contrast to the microprocessors used in personal computers or other general-purpose applications consisting of various discrete chips.

36. Explain Microcontroller with figure. (Architecture of Intel MCS-51 (commonly termed 8051))

Ans:

1. Bus control-

- Data bus-use to transfer 8 bit data.
- Address bus- use to transfer information.

2. Four general purpose parallel i/o port-

- **Port 0 (P0):** address/data bus (AD0-AD7)
- **Port 1 (P1):** A general-purpose 8-bit I/O port.
- **Port 2 (P2):** A general-purpose 8-bit I/O port.
- **Port 3 (P3):** A general-purpose 8-bit I/O port, also used for interrupt requests, timer outputs, and serial communication.

3. Timer and Counter: T0 – T1.

4. Interrupt Control

- By sending S/W instruction.
- By sending H/W signal.

5. Internal RAM and ROM : Data Memory & Program Memory

6. Serial port

7. On-chip Crystal oscillator (OSC)

- 8. **CPU :** Microcontroller 8051 has a central processing unit which is also called ALU (Arithmetic Logic Unit) which performs all arithmetic and logical operation.

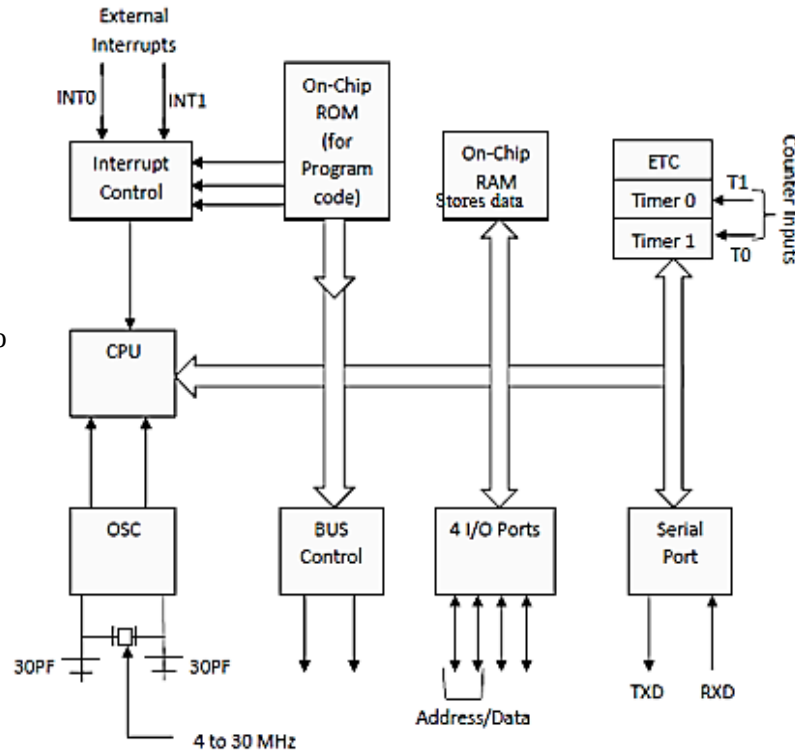


Figure 1: Architecture of Intel MCS-51 (commonly termed 8051).

37. Application of Microcontroller 8051.

- Light sensing and controlling devices like LED.
- Temperature sensing and controlling devices like microwave oven.
- Fire detection and safety devices like fire alarm.
- Measuring devices like volt meters.

10. Interfacing of 8085 with 8255

38. Define 8255.

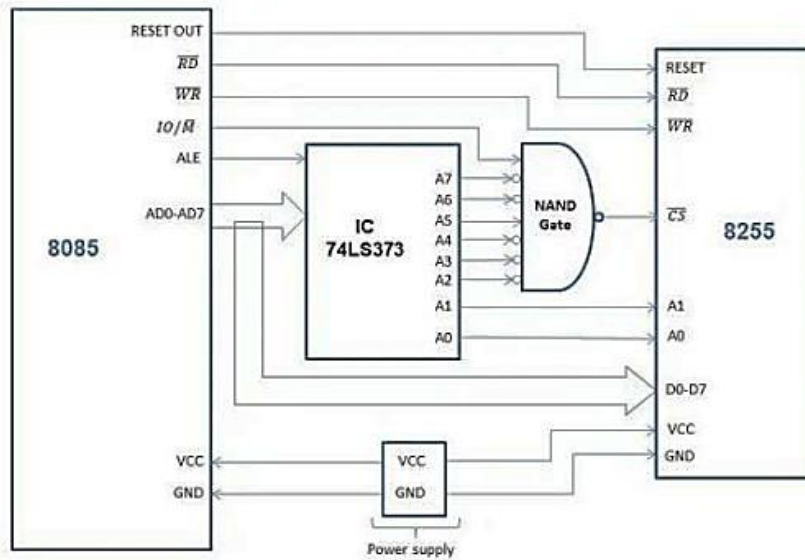
Ans: The 8255 is a programmable peripheral interface. It is an IC used to simplify the interfacing of microprocessors and microcontrollers with I/O devices and increase the number of I/O devices that can be interfaced.

39. Explain Interfacing of 8255 with the 8085 microprocessors.

Ans:

- First of all, we need supply power to both 8085 and 8255 by connecting VCC (Voltage common collector) and GND pins to the appropriate sources.
- RESET OUT pin of 8085 is connected to the RESET pin of 8255.
- RD and WD pins of 8085 are connected to the RD and WD pins of 8255 respectively.
- As we have come to know in this 8085 course, pins AD0-AD7 have dual functionality. They act as both; address bus and data bus.
- After demultiplexing, pins A0 and A1 (from the output of IC74LS373) are connected to the pins A0 and A1 of 8255, respectively
- The last part of making the connections is the chip select logic.

Circuit diagram to connect the 8085 with 8255



40. 8255 Port Address Decoding.

Let us allot the address to the ports A, B, and C of 8255.

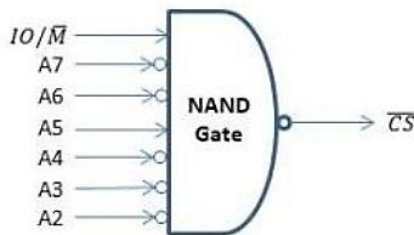
- Port A: 0010 0000 (20H)
- Port B: 0010 0001 (21H)
- Port C: 0010 0010 (22H)
- Control port: 0010 0011 (23H)

The last two bits are A0 and A1. The rest of the 6 bits are used to generate chip select signals.

Table 11-3: 8255 Port Selection

CS	A1	A0	Selects
0	0	0	Port A
0	0	1	Port B
0	1	0	Port C
0	1	1	Control Register
1	x	x	8255 is not selected

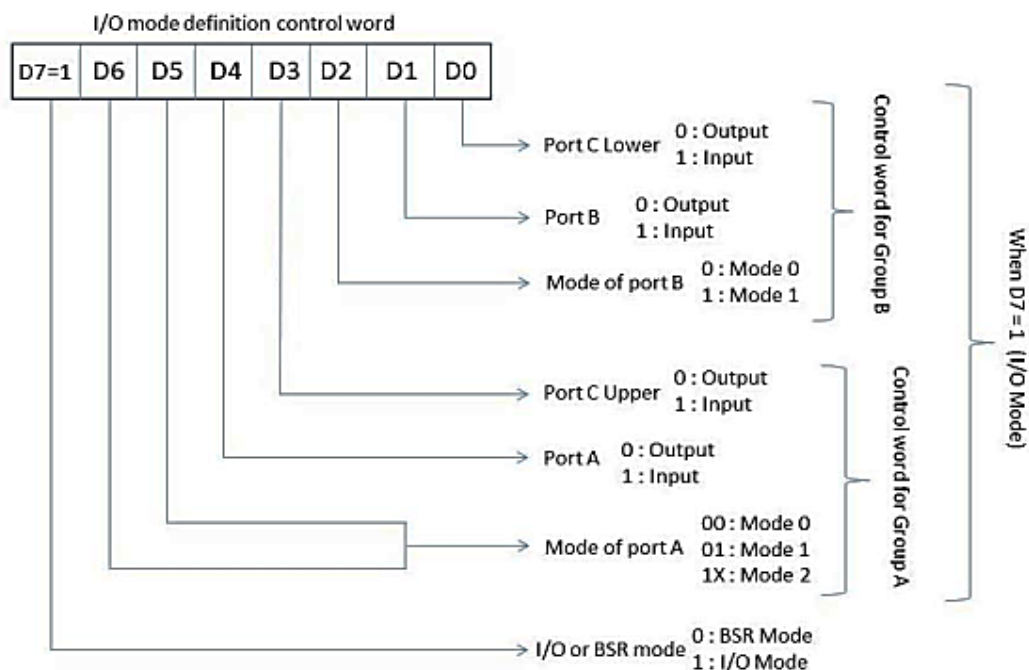
From the above address allocations, we can see that CS should be low only when $A7 = A6 = A4 = A3 = A2 = 0$ and $A5 = 1$. Since we are using the NAND gate to implement chip select logic so, the final circuit of the chip select logic will be 0.



The NAND gate outputs 1 unless all its inputs are 1. So, to get $CS = 0$, all the inputs to the NAND gate must be 1.

41. Consider this example where we are required to take input data from port A and port B and output it at port C or 8255 Control word calculation and step.

Ans:



Step 1:

- Ports will be working in I/O mode and not in BSR mode. So, **D7=1**.
- This is a simple I/O operation. So, we need all three ports A, B, and C to work in mode 0. So, **D6 = D5 = D2 = 0**.
- Port A and B are in input mode and port C is in output mode. Therefore, **D4= D1 = 1** and **D3 = D0 = 0**.

Step 2:

Read input from port A. Input will be stored in the accumulator register (ax). But we need to read another input from port B.

Step 3: Read input from port B.

Step 4: Now, we have the two numbers to be added in Accumulator and register. We add both of them and output the value to port C.

11. 8237 BMA Controller

42. Define DMA Controller.

Ans: Direct memory access (DMA) is a feature of computer systems that allows certain hardware subsystems (such as I/O devices) to access main system memory independently without intervention from CPU.

43. Sort explain Direct Memory Access.

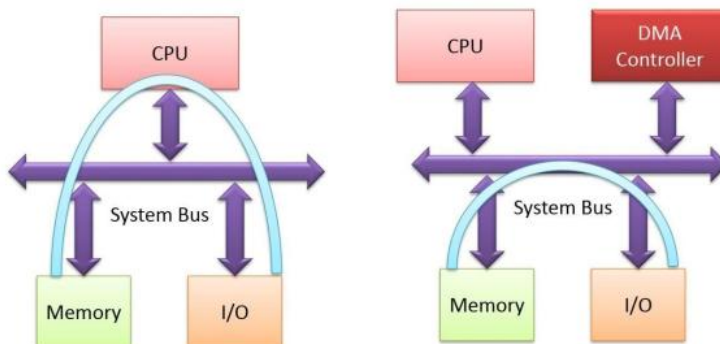
Ans: It is a peripheral core for microprocessor systems. It controls data transfer between the main memory.

- ❖ It provides direct memory access where microprocessor is temporarily disable.
- ❖ We can transfer data very fast.

44. Data Transfer DMA Mode.

Ans:

Data Transfer DMA mode



45. Explain DMA Controller with figure.

DMA Controller interfaces with CPU, I/O Controller, Interrupt Controller and Memory.

HOLD = Hold signal
HOLDA = Acknowledge signal

Handshaking signals between DMA controller and CPU.

DREQ = DMA request signal,
DACK = DMA acknowledge signal,
IORDb = Input-Output read signal,
IOWRb = Input-Output write signal,
EOPb = End of operation signal

Handshaking signals between DMA controller and I/O controller.

MEMRDb = Memory read signal,
MEMWRb = Memory write signal

Handshaking signals between DMA controller and memory.

DMA Controller

